

A SELF-DIRECTED LEARNING COMPANION

Object-Oriented Programming in Java

The complete journey — every pillar explained with real code, visual diagrams, and the exact roadmap to cover it before moving deeper into DSA and Spring Boot.

Encapsulation

Inheritance

Polymorphism

Abstraction

SOLID

Interview-Ready

00 Table of Contents

01 What is OOP & Why Java Cares So Much	3
02 Classes & Objects — The Blueprint Model	4
03 Encapsulation — Guarding the State	5
04 Constructors, this & super	6
05 Inheritance — Building Hierarchies	8
06 Polymorphism — One Name, Many Forms	10
07 Abstraction — Abstract Classes vs Interfaces	12
08 Static vs Instance — The Memory Model	14
09 Access Modifiers — Controlling Visibility	15
10 The Object Class — equals, hashCode, toString	16
11 Composition vs Inheritance — has-a vs is-a	18
12 SOLID Principles — Writing OOP That Scales	19
13 Common Pitfalls & Interview Rapid-Fire	21
14 The Actual Roadmap — Day-by-Day Plan	22
15 Practice Projects to Lock It In	24

01 What is OOP & Why Java Cares So Much

Object-Oriented Programming is a way of organizing code around **objects** — bundles of state (data) and behavior (methods) — instead of around a sequence of instructions acting on loose data. Java was built from the ground up as an OOP-first language: every piece of executable code lives inside a class, there is no such thing as a free-floating function.

OOP rests on four pillars. The rest of this document is essentially a deep dive into each one, but here is the map before you walk into the territory:

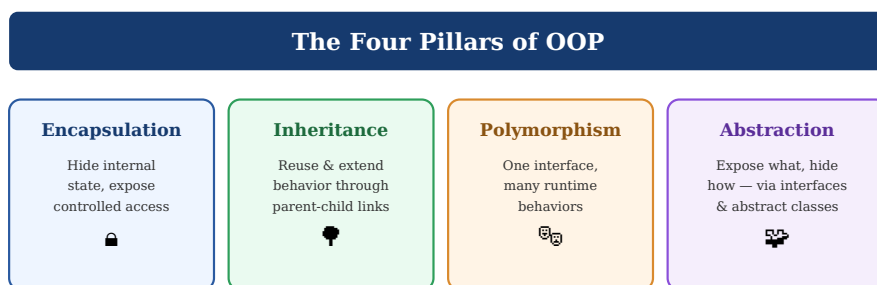


Fig 1.1 — The four pillars, each covered as its own section with runnable examples.

Why this matters for your path

You're building toward SDE placements through Java DSA, so it's tempting to treat OOP as a formality to skim past on the way to arrays and trees. That's a mistake for two concrete reasons:

- **Interviews test it directly.** "Explain polymorphism with an example," "difference between abstract class and interface," "why override equals and hashCode together" are near-guaranteed questions in SDE screens.
- **DSA code quality depends on it.** Clean ``Node``, ``TreeNode``, ``Graph``, ``Comparator`` implementations you'll write in your DSA track are themselves small OOP exercises — this section is not separate from your DSA work, it underpins it.

HOW TO USE THIS DOCUMENT

Read one section a day, type every code example yourself (no copy-pasting into your IDE — this matches your struggle-first approach from your DSA practice), then attempt the mini-exercise at the end of that section before moving on.

02 Classes & Objects — The Blueprint Model

A **class** is a blueprint. An **object** is a concrete thing built from that blueprint, living in memory with its own copy of the instance fields.

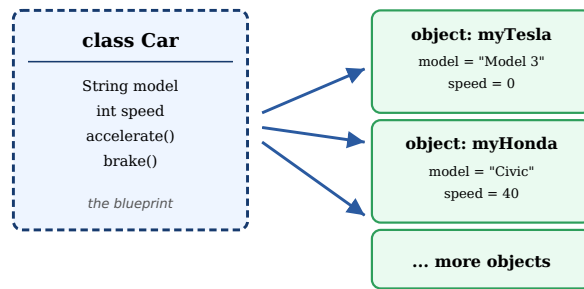


Fig 2.1 — One class, many independent objects, each with its own field values on the heap.

```
public class Car {
    // instance fields — each object gets its own copy
    private String model;
    private int speed;

    // constructor — runs when "new" creates an object
    public Car(String model) {
        this.model = model;
        this.speed = 0;
    }

    public void accelerate(int amount) {
        speed += amount;
        System.out.println(model + " now at " + speed + " km/h");
    }
}

// usage — creating objects from the blueprint
Car myTesla = new Car("Model 3");
Car myHonda = new Car("Civic");
myTesla.accelerate(50);    // Model 3 now at 50 km/h
myHonda.accelerate(40);    // Civic now at 40 km/h — independent state
```

KEY DISTINCTION

The class is compiled once into the `.class` file. Every `new` call allocates a fresh object on the heap with its own field values, but all objects share the same method code from the class — methods aren't duplicated per object.

Mini-exercise

Write a `Student` class with fields `name`, `rollNo`, and an array of 5 `marks`. Add a method `double average()`. Create 3 student objects and print each one's average.

03 Encapsulation — Guarding the State

Encapsulation means keeping fields `private` and exposing only what's necessary through `public` methods (getters/setters, or better — meaningful behavior methods). The goal isn't just "hiding for the sake of hiding" — it's protecting invariants: rules your object must never break.

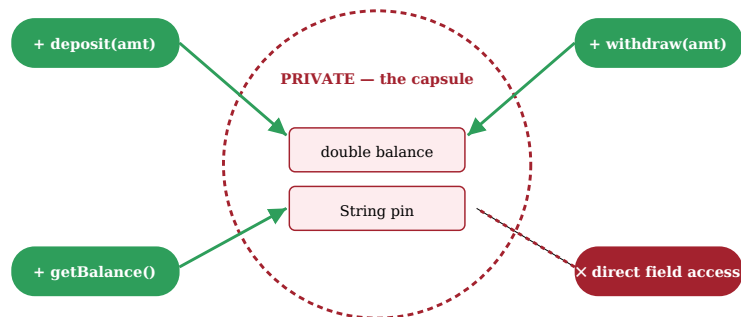


Fig 3.1 — Outside code can only reach the private state through the public methods you define — the "gatekeepers" enforce the rules.

```
public class BankAccount {
    private double balance; // nobody outside can touch this directly

    public void deposit(double amt) {
        if (amt <= 0) throw new IllegalArgumentException("Deposit must be positive");
        balance += amt;
    }

    public void withdraw(double amt) {
        if (amt > balance) throw new IllegalStateException("Insufficient funds");
        balance -= amt;
    }

    public double getBalance() { return balance; } // read-only view out
}
```

If `balance` were `public`, any code anywhere could do `account.balance = -500;` and silently break the invariant "balance never goes negative." Encapsulation is what makes that invariant *enforceable*.

COMMON MISTAKE

Writing a getter and setter for every single field out of habit ("JavaBean-itis") isn't encapsulation — it's just a public field with extra steps. Ask: does this setter enforce a rule, or just pass the value straight through? If it's the latter, consider whether that field should even be mutable from outside.

04 Constructors, this & super

Constructors

A constructor initializes an object's state at creation time. It shares the class name and has no return type — not even `void`.

```
public class Rectangle {
    private double width, height;

    // default constructor
    public Rectangle() { this(1.0, 1.0); } // delegates via this(...)

    // parameterized constructor
    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    public double area() { return width * height; }
}
```

this — resolving ambiguity & chaining

`this` refers to the current object. Its two most common uses:

- **Disambiguation:** `this.width = width;` distinguishes the field from the parameter of the same name.
- **Constructor chaining:** `this(...)` calls another constructor in the same class (must be the first line).

super — reaching into the parent

`super` refers to the immediate parent class. It's used to call the parent's constructor or access a parent method/field that's been overridden or hidden.

```
public class Animal {
    protected String name;
    public Animal(String name) { this.name = name; }
    public void speak() { System.out.println(name + " makes a sound"); }
}

public class Dog extends Animal {
    public Dog(String name) {
        super(name); // MUST call parent constructor first — implicit if omitted & parent has a no-arg ctor
    }

    @Override
    public void speak() {
        super.speak(); // still runs Animal's version too
        System.out.println(name + " barks");
    }
}
```

Construction order: new Dog("Rex")

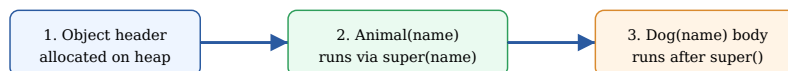


Fig 4.1 — Parent constructors always finish before the child constructor's own body executes.

RULE TO MEMORIZE

If you don't write `super(...)` explicitly, Java inserts a call to the parent's **no-argument** constructor for you. If the parent has no no-arg constructor (because it only defines a parameterized one), your subclass **will not compile** unless you call `super(...)` explicitly.

Mini-exercise

Create a 3-level hierarchy `Vehicle → Car → SportsCar`, each adding one field, each constructor chaining via `super(...)`. Print a message from each constructor and observe the order they fire in when you instantiate a `SportsCar`.

05 Inheritance — Building Hierarchies

Inheritance lets a class (subclass) acquire fields and methods from another class (superclass), modeling an **is-a** relationship. Java supports single inheritance for classes — one direct parent only — but a chain of parents (multilevel) is fine.

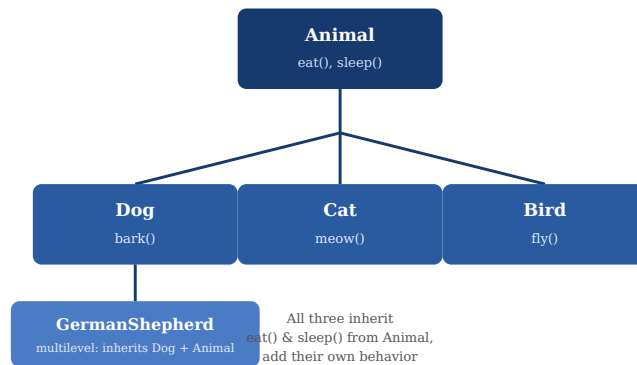


Fig 5.1 — Single inheritance per class, but multilevel chains are allowed (GermanShepherd → Dog → Animal).

```
class Animal {
    void eat() { System.out.println("eating..."); }
}

class Dog extends Animal {    // Dog IS-A Animal
    void bark() { System.out.println("woof!"); }
}

Dog d = new Dog();
d.eat();    // inherited from Animal
d.bark();   // defined in Dog
```

Method overriding basics

A subclass can redefine a parent method with the same signature — this is the doorway into runtime polymorphism, covered fully in Section 6.

```
class Animal {
    void sound() { System.out.println("some generic sound"); }
}

class Cat extends Animal {
    @Override
    void sound() { System.out.println("meow"); }    // overrides parent's version
}
```

WHY JAVA HAS NO MULTIPLE CLASS INHERITANCE

If class **C** could extend both **A** and **B**, and both defined a method `foo()`, the compiler wouldn't know which `foo()` to inherit — the "Diamond Problem." Java sidesteps this for classes entirely, and solves it for interfaces (Section 7) with an explicit resolution rule.

Mini-exercise

Model `Shape → Circle, Rectangle, Triangle`. Give `Shape` an `area()` method returning 0, override it properly in each subclass, then loop over an array of `Shape` references holding different subclass objects and print each area.

06 Polymorphism — One Name, Many Forms

Polymorphism means the same method name behaves differently depending on context. Java gives you two distinct flavors — don't blur them, interviewers specifically probe the difference.

Compile-time (Overloading)	Runtime (Overriding)
Same method name, different parameter list, same class	Same signature, redefined in a subclass
Resolved by the compiler based on the arguments you pass — a.k.a. static binding	Resolved at runtime based on the object's actual type — a.k.a. dynamic binding
<code>add(int,int)</code> VS <code>add(double,double)</code>	<code>Animal a = new Dog(); a.sound();</code> → calls Dog's version

Overloading example

```
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}

// The compiler decides WHICH add() to call by matching argument types/count at compile time
```

Overriding & dynamic dispatch — the classic interview diagram

```
class Animal {
    void sound() { System.out.println("generic sound"); }
}
class Dog extends Animal {
    @Override void sound() { System.out.println("woof"); }
}
class Cat extends Animal {
    @Override void sound() { System.out.println("meow"); }
}

Animal a = new Dog(); // reference type Animal, OBJECT type Dog
a.sound();             // prints "woof" — decided by the OBJECT, not the reference
```

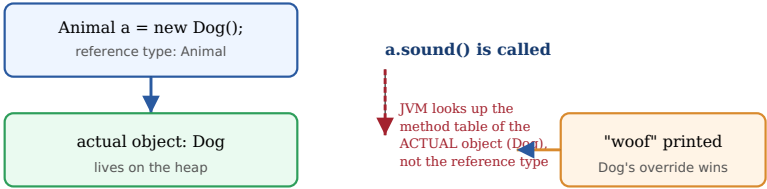


Fig 6.1 — Dynamic dispatch: the JVM always resolves an overridden method using the object's real type at runtime.

OVERLOAD RESOLUTION RULES (COMPILE-TIME)

Java picks the overload in this priority order: (1) exact match, (2) widening primitive conversion (int→long), (3) autoboxing (int→Integer), (4) varargs. If two overloads are equally specific, you'll get a compile error, not a runtime surprise.

Mini-exercise

Write a `Shape` hierarchy where `Shape` has `draw()`, overridden in `Circle` and `Square`. Store them in a single `List<Shape>`, loop through, and call `draw()` on each — verify each prints its own version despite the list being typed as `Shape`.

07 Abstraction — Abstract Classes vs Interfaces

Abstraction means exposing *what* an object does while hiding *how*. Java gives you two tools: **abstract classes** and **interfaces**. Confusing when to use which is one of the most common gaps at your stage — this section draws the line clearly.

Aspect	Abstract Class	Interface
Keyword	<code>abstract class</code>	<code>interface</code>
Fields	Any (instance state allowed)	Only <code>public static final</code> constants
Method bodies	Mix of abstract + concrete methods	Abstract by default; <code>default</code> / <code>static</code> methods allowed (Java 8+)
Constructors	Yes	No
Inheritance	A class extends only one abstract class	A class can implement multiple interfaces
Use when	Subclasses share common state/behavior — an "is-a" family with a real base identity	Unrelated classes need to guarantee the same capability — a "can-do" contract

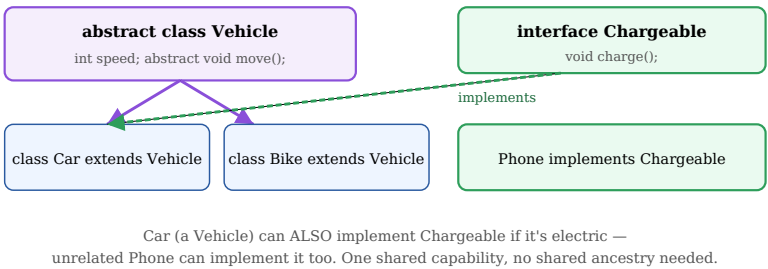


Fig 7.1 — Abstract class = shared identity & state. Interface = shared capability across unrelated classes.

```
abstract class Vehicle {
    protected int speed;
    abstract void move(); // no body — subclass MUST implement
    void honk() { System.out.println("beep"); } // shared concrete behavior
}

interface Chargeable {
    void charge(); // implicitly public abstract
    default void showBattery() { // Java 8+: default method with a body
        System.out.println("battery ok");
    }
}

class ElectricCar extends Vehicle implements Chargeable {
    @Override void move() { System.out.println("gliding silently"); }
    @Override public void charge() { System.out.println("charging..."); }
}
```

INTERVIEW ONE-LINER WORTH MEMORIZING

"Use an abstract class when subclasses share code and state and belong to the same family tree. Use an interface when unrelated classes need to promise the same capability — Java allows implementing many interfaces but extending only one class."

Mini-exercise

Create `interface Flyable { void fly(); }` and `interface Swimmable { void swim(); }`. Make a `Duck` class implement both, alongside extending an abstract `Animal` class. This demonstrates multiple interface implementation with single class inheritance.

08 Static vs Instance — The Memory Model

`static` members belong to the **class itself** — one shared copy. Instance members belong to **each object** — a separate copy per object. Understanding where each lives in memory removes a lot of confusion about when to use `static`.

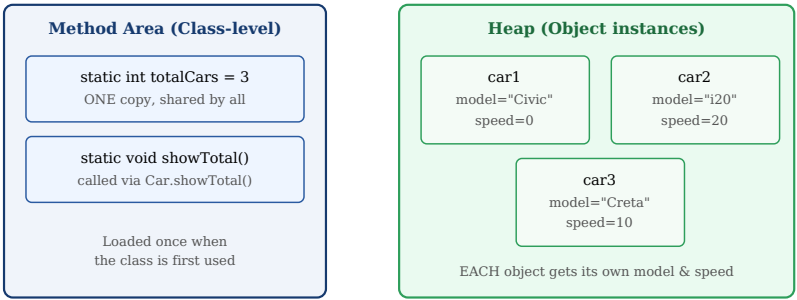


Fig 8.1 — Static members live once in the method area; instance fields live once per object on the heap.

```
class Car {
    static int totalCars = 0;    // ONE copy shared across all Car objects
    String model;               // separate copy per object

    Car(String model) {
        this.model = model;
        totalCars++;           // every construction bumps the shared counter
    }

    static void printTotal() {
        System.out.println("Total cars made: " + totalCars);
        // NOTE: static methods cannot access instance fields like "model" directly -
        // there's no specific object in context to take "model" from
    }
}
```

Use static for	Use instance for
Counters/shared state across all objects	Data that differs per object
Utility methods with no object state needed (<code>Math.sqrt()</code>)	Behavior that depends on that object's own fields
Constants (<code>static final</code>)	Anything tied to "this particular instance"

CLASSIC GOTCHA

A static method cannot directly reference `this` or any instance field/method, because it can be called without any object existing at all (`ClassName.method()`). This is a frequent early compile error — "non-static variable cannot be referenced from a static context."

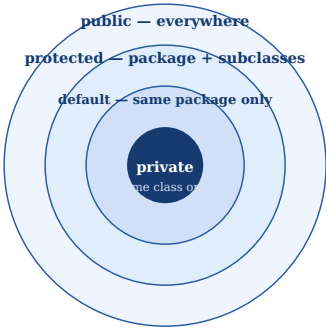


Fig 9.1 — Visibility widens outward: private → default (package-private) → protected → public.

Modifier	Same Class	Same Package	Subclass (diff package)	Everywhere
private	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

PRACTICAL GUIDELINE

Default to `private` for fields always. Use `protected` only for members a subclass genuinely needs to extend or override. Reach for `public` only on the methods that make up your class's actual contract with the outside world — everything else stays as hidden as possible.

10 The Object Class — equals, hashCode, toString

Every class in Java implicitly extends `Object`, even if you don't write `extends Object`. That gives you three methods worth mastering, because getting them wrong silently breaks collections like `HashSet` and `HashMap`.

toString()

```
class Point {
    int x, y;
    @Override public String toString() {
        return "Point(" + x + ", " + y + ")";
    }
}

// Without overriding, System.out.println(point) prints something like Point@1b6d3586 (hash-based, useless)
```

equals() and hashCode() — the contract

The default `equals()` from `Object` checks reference identity (are these the exact same object in memory?), not logical equality. If you override `equals()`, you **must** override `hashCode()` too — otherwise two "equal" objects can land in different hash buckets and a `HashSet` will treat them as different.

```
class Point {
    int x, y;

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point)) return false;
        Point p = (Point) o;
        return x == p.x && y == p.y;
    }

    @Override public int hashCode() {
        return Objects.hash(x, y); // MUST be consistent with equals()
    }
}
```

Why the contract matters: HashSet lookups

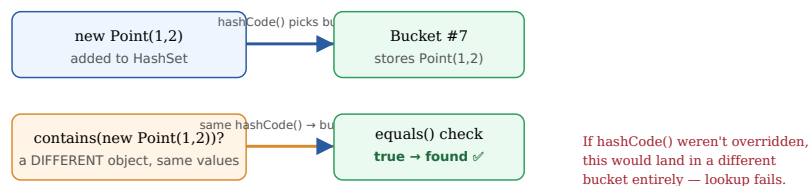


Fig 10.1 — equals/hashCode must move together, or hash-based collections silently fail to find "equal" objects.

INTERVIEW TRAP

"Can two unequal objects have the same hashCode()?" — Yes, this is a hash collision, and it's fine. "Can two equal objects have different hashCode()?" — No, this breaks the contract and is a bug.

11 Composition vs Inheritance — has-a vs is-a

Inheritance models **is-a**. Composition models **has-a** — one class holds a reference to another and delegates work to it. Modern OOP design leans toward "*favor composition over inheritance*" because inheritance creates tight coupling to a parent's implementation, while composition stays flexible.

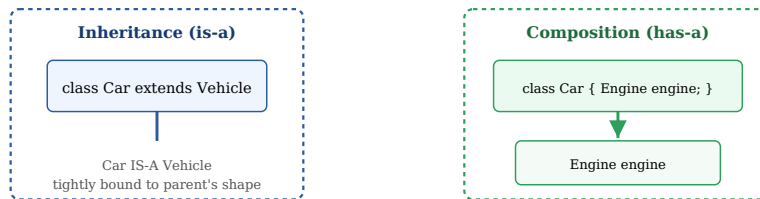


Fig 11.1 — Composition lets Car swap in a different Engine at runtime; inheritance locks Car into whatever Vehicle defines.

```
// Composition: Car HAS-A Engine, delegates work to it
class Engine {
    void start() { System.out.println("Engine roars to life"); }
}

class Car {
    private final Engine engine;           // composed-in, not inherited

    Car(Engine engine) { this.engine = engine; } // can inject ANY engine implementation

    void start() {
        engine.start();                    // delegates instead of inheriting behavior
        System.out.println("Car is ready to drive");
    }
}
```

DECISION RULE

Ask: "Is B fundamentally a specialized kind of A, following A's contract everywhere?" → inheritance. "Does B just need to use A's capability internally without being one?" → composition. When in doubt, composition is the safer default in real codebases.

12 SOLID Principles — Writing OOP That Scales

SOLID is five design principles that keep OOP code maintainable as it grows. You won't need to recite formal definitions in early interviews, but recognizing violations in a code snippet is a common ask.

Letter	Principle	In one line
S	Single Responsibility	A class should have exactly one reason to change.
O	Open/Closed	Open for extension, closed for modification — add new behavior via new classes, not by editing existing ones.
L	Liskov Substitution	A subclass object must be usable anywhere its parent type is expected, without breaking correctness.
I	Interface Segregation	Prefer several small, focused interfaces over one bloated one clients are forced to implement in full.
D	Dependency Inversion	Depend on abstractions (interfaces), not concrete classes — enables swapping implementations.

A quick Liskov violation to recognize

```
class Rectangle {
    protected int width, height;
    void setWidth(int w) { width = w; }
    void setHeight(int h) { height = h; }
    int area() { return width * height; }
}

// Tempting but WRONG: Square "is-a" Rectangle mathematically, but...
class Square extends Rectangle {
    @Override void setWidth(int w) { width = height = w; }    // breaks caller's expectation!
    @Override void setHeight(int h) { width = height = h; }
}

// Code written against Rectangle that calls setWidth(5); setHeight(4); expects area()==20.
// Pass a Square in and area() silently becomes 16 — substitution breaks the contract.
```

WHY THIS MATTERS RIGHT NOW

SOLID reasoning is exactly what separates "I can write a class" from "I can design a system" — the gap that shows up in mid-to-senior SDE interviews and in your own Spring Boot backend work later.

13 Common Pitfalls & Interview Rapid-Fire

Pitfalls to avoid while practicing

- **Overusing inheritance** for code reuse alone — if it's not a genuine is-a relationship, use composition.
- **Forgetting** `@Override` — without it, a typo in the method signature silently creates a new overload instead of overriding, and you won't know until runtime behavior looks wrong.
- **Public mutable fields** — defeats encapsulation; any caller can corrupt state.
- **Overriding** `equals()` **without** `hashCode()` — breaks hash-based collections silently.
- **Deep inheritance chains** (4+ levels) — hard to reason about, fragile to change.
- **Calling overridable methods from a constructor** — the subclass's override can run before the subclass has finished initializing its own fields.

Rapid-fire Q&A cheat sheet

Question	Crisp Answer
Can a constructor be private?	Yes — used in Singleton pattern to block external instantiation.
Can we override a static method?	No — static methods are hidden, not overridden (no dynamic dispatch); this is "method hiding."
Can an abstract class have a constructor?	Yes — called via <code>super()</code> when a subclass is instantiated, even though you can't <code>new</code> the abstract class directly.
Can an interface have a constructor?	No — interfaces have no state to initialize.
Difference between overloading and overriding, one line each?	Overloading = same name, different params, same class, compile-time. Overriding = same signature, subclass, runtime.
What is the diamond problem and how does Java avoid it for interfaces?	Ambiguity when two parents define the same method. For interfaces, if two default methods clash, the implementing class MUST override and resolve it explicitly.
Why favor composition over inheritance?	Looser coupling, easier to change implementation without breaking the "has-a" class, avoids fragile base class problems.
What does "programming to an interface" mean?	Declare variables/parameters with the interface type (<code>List<T> l = new ArrayList<>()</code>) so the concrete implementation can be swapped freely.

14 The Actual Roadmap — Day-by-Day Plan

This is designed as a focused **12-day OOP sprint**, roughly 1.5-2 hours/day, meant to sit inside your existing Java DSA track rather than replace it — treat it as the foundation layer underneath the patterns you're already drilling.

Day	Hours	Focus	Deliverable
D1	1.5h	Classes, objects, the `new` keyword, memory model basics (Sec 1-2)	Student class exercise, working & tested
D2	1.5h	Encapsulation, invariants, access-control mindset (Sec 3)	BankAccount with deposit/withdraw validation
D3	2h	Constructors, this, super, constructor chaining (Sec 4)	3-level Vehicle→Car→SportsCar hierarchy
D4	2h	Inheritance fundamentals, method overriding basics (Sec 5)	Shape hierarchy with polymorphic area()
D5	2h	Polymorphism deep-dive: overload resolution + dynamic dispatch (Sec 6)	List<Shape> draw-all exercise
D6	1.5h	Review D1-D5 cold — rebuild 2 exercises from memory, no notes	Self-graded rebuild log
D7	2h	Abstraction: abstract classes vs interfaces, default methods (Sec 7)	Duck implementing Flyable + Swimmable
D8	1.5h	Static vs instance, memory model, common static gotchas (Sec 8)	Car counter using static field
D9	1h	Access modifiers — visibility rules cold recall (Sec 9)	Fill the 4x4 visibility table from memory
D10	2h	equals/hashCode/toString contract + HashSet behavior (Sec 10)	Point class working correctly inside a HashSet
D11	2h	Composition vs inheritance, intro to SOLID (Sec 11-12)	Rewrite one earlier inheritance exercise using composition
D12	2h	Full rapid-fire Q&A pass (Sec 13) + build one capstone project (Sec 15)	Capstone project committed to GitHub



Fig 14.1 — 12-day sprint timeline: build → cold-review checkpoint → build → capstone.

~21 focused hours total 1 cold-recall checkpoint (D6) 1 capstone project (D12)

FITTING IT INTO YOUR BIGGER PICTURE

Run this sprint in parallel with your existing daily DSA habit, not instead of it — 1.5-2h for OOP, keep your regular DSA slot untouched. Once D12 is done, the OOP vocabulary here directly feeds your Spring Boot learning (classes as `@Service`/`@Repository`, interfaces for dependency inversion) and makes your `TreeNode`/`Graph`/`Comparator` DSA code noticeably cleaner.

15 Practice Projects to Lock It In

Reading and small exercises build recognition. These slightly larger builds force independent recall and design decisions — pick **one** as your Day 12 capstone.

Option A — Library Management System (console-based)

- `abstract class Item` → `Book`, `Magazine`, `DVD` (inheritance + abstraction)
- `interface Borrowable { void borrow(); void returnItem(); }` implemented selectively
- `Library` class composed of a `List<Item>` (composition)
- Override `equals()` / `hashCode()` on `Item` by ISBN so a `HashSet<Item>` correctly detects duplicates
- A `static int totalItemsInLibrary` counter

Option B — Employee Payroll System

- `abstract class Employee` with `abstract double calculateSalary()`
- `FullTimeEmployee`, `Contractor`, `Manager` extends `FullTimeEmployee` (multilevel inheritance)
- `interface Taxable { double calculateTax(); }`
- Polymorphic payroll run: loop a `List<Employee>` and call `calculateSalary()` on each
- SOLID checkpoint: keep tax calculation logic in its own class, not stuffed inside `Employee` (Single Responsibility)

Option C — Shape/Geometry Toolkit (lighter build, good for a tighter timeline)

- `Shape` abstract class with `area()`, `perimeter()`
- `Circle`, `Rectangle`, `Triangle`, `Square` extends `Rectangle` — deliberately hit the Liskov pitfall from Section 12 and then fix it by NOT inheriting `Square` from `Rectangle`
- `Comparable<Shape>` implemented to sort shapes by area — ties directly back into your DSA sorting/Comparator work

DEFINITION OF DONE FOR WHICHEVER YOU PICK

Pushed to your GitHub, README explaining which OOP concept each class demonstrates, and you can explain out loud — without looking at the code — why each design choice (inheritance vs interface vs composition) was made where it was made.